

BioMechAI — Module 02: Real-Time 3D Anatomical Pose Detection & On-Device Computer Vision Pipeline

TABLE OF CONTENTS & RAPID-LOOKUP INDEX

1. Executive Summary & Core Mission
2. Real-World Computer Vision Challenge: Why Mobile Pose Detection is Difficult
3. Step-by-Step Chronological Engineering Evolution
4. Deep Learning Architecture: BlazePose Two-Stage Detector-Tracker Pipeline
4.1 Stage 1: The Face/Torso ROI Alignment Detector
4.2 Stage 2: The 33-Landmark 3D Coordinate Regression Network
4.3 Tracking Optimization: Bypassing the Detector in Consecutive Frames
5. The 33 Anatomical Keypoint Topology & 3D Spatial Coordinate System
5.1 Landmark Semantic Mapping & kSkeletonPairs Graph
5.2 The 3D Coordinate System: Understanding Relative Depth (\$z\$) and Visibility
6. High-Performance Camera Stream Ingestion Pipeline (YUV420 Memory Assembly)
7. Mathematical Joint Trigonometry & Planar Angle Derivations
8. Exponential Moving Average (EMA) Coordinate Smoothing & Jitter Elimination
9. Downstream System Feed Pipeline (How Module 02 Powers Modules 3, 4, 5, 6, & 7)
10. Scientific Standards, Google Research Benchmarks, & Literature Citations
11. Panel Defense Quick-Reference: Frequently Asked Questions & Rapid Answers

1. Executive Summary & Core Mission

Module 02 is the sensory visual foundation of the entire BioMechAI system. Before any artificial intelligence model can classify an exercise (Module 03), count a repetition (Module 04), grade posture correctness (Module 05), measure body levers (Module 06), or warn against a ligament tear (Module 07), the system must first perceive where the human body is located in physical space.

Plain-English Concept: Imagine a professional athlete wearing an expensive Hollywood motion-capture suit covered in dozens of glowing ping-pong balls while dozens of infrared laboratory cameras track their movement. Module 02 achieves that exact same motion-capture capability using *zero wearable sensors, zero special suits, and zero external hardware*. Using nothing more than the standard single-lens camera built into a regular smartphone, Module 02 identifies 33 anatomical landmarks across the athlete's body 30 times every second, transforming raw camera video into a living, three-dimensional digital skeleton.

The module delivers:

- **33 Three-Dimensional Anatomical Keypoints:** Pinpoints exact coordinates for the nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles, heels, and toes.
 - **3D Relative Depth Perception (z -axis):** Infers the relative depth of each joint in camera space, solving perspective foreshortening.
 - **High-Speed On-Device Inference (30 FPS):** Executes locally on the smartphone's Neural Processing Unit (NPU) and GPU in $\sim 18\text{--}28$ milliseconds per frame, consuming zero cloud bandwidth and operating completely offline.
 - **Zero-Copy Memory Stream Pipeline:** High-performance YUV420 multi-plane memory assembly via Flutter's `WriteBuffer`, preventing memory leaks and frame stuttering.
-

2. Real-World Computer Vision Challenge: Why Mobile Pose Detection is Difficult

Extracting human skeletal pose in real time on mobile hardware presents severe computational hurdles:

1. Mobile Silicon Thermal & Compute Constraints:

Desktop computer vision models (such as OpenPose or AlphaPose) require massive desktop GPUs consuming $\$250\text{--}400$ Watts of electrical power. Running such models on a mobile smartphone processor would drain the battery in 15 minutes, cause extreme thermal throttling, and drop the frame rate to an unusable $\$2\text{--}4$ FPS.

2. Severe Anatomical Occlusion During Exercise:

In fitness training, limbs constantly cross in front of or behind each other. During a side-profile squat or

push-up, the near leg or arm completely covers (occludes) the far limb. Traditional object detectors fail when parts of the target object vanish from sight.

3. **High-Frequency Spatial Jitter:**

Because mobile camera sensors have small lenses, low light causes sensor noise that causes detected joint coordinates to shake erratically by several pixels between consecutive frames. If fed directly into biomechanical velocity formulas, this artificial vibration creates catastrophic errors in rep counting.

4. **YUV420 Camera Format Incompatibility:**

Android smartphone cameras stream video frames in the raw **YUV420** multi-plane format (Y = luminance/brightness, U/V = chrominance/color), whereas deep learning vision models expect continuous planar RGB or NV21 byte arrays. Converting every frame using naive nested loops locks the mobile UI thread and crashes the app.

Module 02 overcomes all of these obstacles through an optimized two-stage pipeline, hardware-accelerated memory assembly, and exponential temporal smoothing.

3. Step-by-Step Chronological Engineering Evolution

The pose detection architecture evolved through three rigorous engineering phases between FYP-I and the final FYP-II release:

Phase 1: Cloud-Based Python OpenPose (FYP-I Early Exploration)

In the early prototyping phase of FYP-I, the team tested streaming mobile video frames over HTTP to a cloud server running CMU OpenPose.

Critical Failure Discovered: Streaming 1080p video frames over mobile Wi-Fi created >400ms of round-trip network latency. Frame rates stalled at 6-8 FPS, cellular data quotas were instantly exhausted, and any Wi-Fi hiccup froze the entire pose tracking pipeline. The team determined that cloud pose estimation is fundamentally unviable for real-time athletic coaching.

Phase 2: On-Device Single-Stage Pose Estimation

The pipeline was brought entirely on-device by compiling TensorFlow Lite models (MoveNet SinglePose). While inference time improved to ~45ms, MoveNet was limited to only 17 keypoints and lacked foot and hand landmarks (heels and toes were absent), which prevented accurate knee valgus tracking and foot placement validation.

Phase 3: Google ML Kit BlazePose Integration (Semester 8 Baseline)

In the final FYP-II production release, the team implemented **Google ML Kit BlazePose** in `pose_detection_service.dart`:

- Upgraded from 17 to **33 full-body anatomical landmarks** (adding heels, foot indices, and thumbs).

- Implemented the two-stage detector-tracker pipeline configured for high-speed streaming mode (`PoseDetectionMode.stream`).
 - Engineered the zero-copy `WriteBuffer` YUV420 memory assembler, achieving sustained **30 FPS** on consumer Android hardware.
 - Built a 90-frame temporal landmark circular buffer (`landmarkBuffer`) providing a rolling 3-second memory window for downstream action recognition models.
-

4. Deep Learning Architecture: BlazePose Two-Stage Pipeline

Google BlazePose (Bazarevsky et al., Google Research 2020) achieves real-time mobile speed by separating the problem of finding a human from the problem of tracking their joints:

```
THE BLAZEPOSE TWO-STAGE DETECTOR-TRACKER ARCHITECTURE
=====
[VIDEO FRAME T = 1] (Initial Discovery)
|
v
STAGE 1: Heavy Pose Detector Network
* Scans full video frame (1280x720)
* Detects facial features & mid-hip midpoint
* Computes human bounding box & rotation angle
|
v
STAGE 2: Lightweight 33-Landmark Regression Network
* Cropped Region of Interest (ROI) (256x256 tensor)
* Predicts 33 3D Spatial Landmarks: (x, y, z, visibility)
* Derives tracking bounding box for Frame T = 2
|
v
[VIDEO FRAME T = 2, 3, 4...] (Consecutive Streaming Frames)
|
+----> BYPASS STAGE 1 ENTIRELY! (Zero full-frame scanning)
|
+----> Crop new ROI directly from previous frame's predicted landmarks
|
v
STAGE 2: Lightweight 33-Landmark Regression Network (Only takes ~18 ms!)
* If tracking confidence drops below threshold: Re-awaken Stage 1 Detector.
=====
```

4.1 Stage 1: The Face/Torso ROI Alignment Detector

Finding a full human body in an unconstrained room is computationally expensive. However, the human head and face are rigid, highly distinct visual structures with minimal geometric deformation compared to arms and legs. BlazePose's Stage 1 detector first identifies the face and mid-hip vector. From these anchor points, it projects an aligned, rotated bounding box around the body.

4.2 Stage 2: The 33-Landmark 3D Regression Network

The aligned bounding box is cropped, scaled down to a compact 256×256 tensor, and passed to a lightweight heat-map regression convolutional neural network. The network outputs:

- **33 Planar Coordinates (x, y)** : Normalized screen positions in range $[0.0, 1.0]$.
- **33 Relative Depth Coordinates (z)** : Estimated distance along the camera optical axis.
- **33 Visibility Likelihoods**: Confidence values $[0.0, 1.0]$ representing whether the joint is genuinely visible or hidden behind another body part.

4.3 Tracking Optimization: Bypassing the Detector

This is the key architectural breakthrough: **Stage 1 executes only on the very first frame of a workout**. In all subsequent frames, the system uses the joints predicted in the previous frame to mathematically project where the athlete's body must be in the current frame. This eliminates 80% of computational overhead, allowing the lightweight tracker to run continuously at 30 FPS without heating up the phone.

5. The 33 Anatomical Keypoint Topology & 3D Coordinate System

BlazePose extracts 33 standardized anatomical keypoints mapped across the human kinetic chain:

BLAZEPOSE 33-LANDMARK TOPOLOGY MAPPING

=====

HEAD: 0: Nose | 1: Left Eye Inner | 2: Left Eye | 3: Left Eye Outer

4: Right Eye Inner | 5: Right Eye | 6: Right Eye Outer

7: Left Ear | 8: Right Ear | 9: Mouth Left | 10: Mouth Right

UPPER BODY: 11: Left Shoulder | 12: Right Shoulder

13: Left Elbow | 14: Right Elbow

15: Left Wrist | 16: Right Wrist

17: Left Pinky | 18: Right Pinky

19: Left Index | 20: Right Index

21: Left Thumb | 22: Right Thumb

LOWER BODY: 23: Left Hip | 24: Right Hip

25: Left Knee | 26: Right Knee

27: Left Ankle | 28: Right Ankle

29: Left Heel | 30: Right Heel

31: Left Foot Index | 32: Right Foot Index

=====

5.1 The kSkeletonPairs Bone Connectivity Graph

In `pose_detection_service.dart`, bone connections are codified as a directed skeletal graph connecting adjacent physiological levers:

```
const List<List<PoseLandmarkType>> kSkeletonPairs = [
  [PoseLandmarkType.leftShoulder, PoseLandmarkType.rightShoulder],
  [PoseLandmarkType.leftShoulder, PoseLandmarkType.leftElbow],
  [PoseLandmarkType.leftElbow, PoseLandmarkType.leftWrist],
  [PoseLandmarkType.rightShoulder, PoseLandmarkType.rightElbow],
  [PoseLandmarkType.rightElbow, PoseLandmarkType.rightWrist],
  [PoseLandmarkType.leftShoulder, PoseLandmarkType.leftHip],
  [PoseLandmarkType.rightShoulder, PoseLandmarkType.rightHip],
  [PoseLandmarkType.leftHip, PoseLandmarkType.rightHip],
  [PoseLandmarkType.leftHip, PoseLandmarkType.leftKnee],
  [PoseLandmarkType.leftKnee, PoseLandmarkType.leftAnkle],
  [PoseLandmarkType.rightHip, PoseLandmarkType.rightKnee],
  [PoseLandmarkType.rightKnee, PoseLandmarkType.rightAnkle],
  [PoseLandmarkType.leftAnkle, PoseLandmarkType.leftHeel],
  [PoseLandmarkType.rightAnkle, PoseLandmarkType.rightHeel],
  [PoseLandmarkType.leftHeel, PoseLandmarkType.leftFootIndex],
  [PoseLandmarkType.rightHeel, PoseLandmarkType.rightFootIndex],
];
```

5.2 The 3D Coordinate System: Understanding Relative Depth (\$z\$)

Unlike naive 2D vision models, every landmark contains three spatial components:

$\mathbf{L}_k = (x_k, y_k, z_k, v_k)$ Where:

- $x_k \in [0.0, 1.0]$: Horizontal position normalized across camera frame width.
- $y_k \in [0.0, 1.0]$: Vertical position normalized across camera frame height.
- z_k : **Estimated relative depth** along the camera's optical z-axis. The coordinate origin ($z=0$) is calibrated to the midpoint between the athlete's hips. A negative z indicates the joint is closer to the camera than the hips; a positive z indicates the joint is further away.
- $v_k \in [0.0, 1.0]$: Probability that the landmark is within the frame and un-occluded.

Plain-English Concept: If you are standing facing the camera and punch your right fist directly forward toward the phone screen, your right wrist's x and y on the screen might barely change. But its z coordinate plunges sharply into negative numbers, telling the computer that your arm has extended forward in 3D space!

6. High-Performance Camera Stream Ingestion Pipeline

A frequent cause of mobile app crashes is improper image memory handling. In Android, the camera hardware writes raw image frames into three separate memory planes: Plane 0 (Y Luminance), Plane 1 (U Chrominance), and Plane 2 (V Chrominance).

Module 02 implements a high-throughput, zero-copy buffer assembler in `processFrame()` :

```
// High-Performance Camera Stream Processing (pose_detection_service.dart)
final WriteBuffer allBytes = WriteBuffer();
for (final Plane plane in image.planes) {
  allBytes.putUint8List(plane.bytes);
}
final bytes = allBytes.done().buffer.asUint8List();

final InputImageMetadata metadata = InputImageMetadata(
  size: Size(image.width.toDouble(), image.height.toDouble()),
  rotation: InputImageRotationValue.fromRawValue(camera.sensorOrientation)
    ?? InputImageRotation.rotation0deg,
  format: InputImageFormatValue.fromRawValue(image.format.raw)
    ?? InputImageFormat.yuv420,
  bytesPerRow: image.planes[0].bytesPerRow,
);

final inputImage = InputImage.fromBytes(bytes: bytes, metadata: metadata);
final poses = await _poseDetector.processImage(inputImage);
```

Defensive Pipeline Safeguards:

1. **Concurrency Mutex Lock (`_isBusy`)**: If the smartphone's NPU takes 22 ms to process a frame while the camera delivers a new frame after 16 ms , the incoming frame is immediately dropped without queue buildup, preventing memory spikes.
2. **Hardware Sensor Orientation Mapping**: Automatically maps Android's hardware sensor orientation (90° , 270°) to upright Cartesian coordinates, preventing inverted or sideways skeletons.
3. **Temporal Landmark Ring Buffer (90 Frames)**: Maintains the latest 90 frames of full-body coordinates in memory (`landmarkBuffer`), providing a continuous 3-second temporal sliding window for Module 03 action recognition.

7. Mathematical Joint Trigonometry & Planar Angle Derivations

To compute real-time joint angles for exercise form evaluation, `pose_detection_service.dart` implements the two-dimensional arctangent formulation:

Planar Joint Angle Formulation (jointAngle method):

Let vertex joint B connect adjacent landmarks A and C (e.g., Shoulder A , Elbow B , Wrist C):

1. Calculate the directional vector angles relative to the horizontal axis: $\theta_1 = \text{atan2}(y_C - y_B, x_C - x_B)$ $\theta_2 = \text{atan2}(y_A - y_B, x_A - x_B)$
2. Compute the absolute angular difference: $\Delta \theta = |\theta_1 - \theta_2| \cdot \frac{180^\circ}{\pi}$
3. Normalize to the interior geometric angle $[0^\circ, 180^\circ]$: $\theta = \begin{cases} \Delta \theta & \text{if } \Delta \theta \leq 180.0^\circ \\ 360.0^\circ - \Delta \theta & \text{if } \Delta \theta > 180.0^\circ \end{cases}$

8. Exponential Moving Average (EMA) Coordinate Smoothing

To eliminate jitter and camera sensor noise, all detected landmark coordinates pass through an Exponential Moving Average (EMA) filter in `skeletonPainter.dart` before lines are drawn on screen:

EMA Temporal Smoothing Formulation:

$$\mathbf{P}_{\text{smoothed}}(t) = 0.70 \cdot \mathbf{P}_{\text{target}}(t) + 0.30 \cdot \mathbf{P}_{\text{prev}}(t-1)$$
 Where $\mathbf{P}_{\text{target}}(t)$ is the incoming raw coordinate and $\mathbf{P}_{\text{prev}}(t-1)$ is the previous smoothed coordinate.

Occlusion Clamping: If a joint's detection likelihood drops below \$0.45\$ (e.g., hidden behind back or blocked by weights), it is automatically removed from the smoothing buffer (`_smoothedPoints.remove(type)`), preventing distorted phantom lines.

9. Downstream System Feed Pipeline

Module 02 is the single source of visual truth across the entire BioMechAI architecture. The table below illustrates how Module 02 feeds the remaining modules:

Downstream Module	Data Fed from Module 02	How It Is Utilized
Module 03: Action Recognition	48-frame sliding window from <code>landmarkBuffer</code>	Transformed into 3D spatiotemporal connected limb heatmap tensors for PoseC3D SlowOnly ResNet-50.
Module 04: Rep Counting	Instantaneous Knee & Elbow angles	Drives the 4-stage closed finite state machine (TOP $\rightarrow$ DESCENDING $\rightarrow$ BOTTOM $\rightarrow$ ASCENDING $\rightarrow$ TOP).
Module 05: Posture Correctness	33 3D Spatial Landmarks	Calculates 3D vector dot products, computes form scores (\$0-100\%\$), and colors skeleton Green vs. Red.
Module 06: Body Scanner	Ankle-to-nose vertical pixel span	Resolves monocular scale ambiguity ($scale = H_{\text{cm}} / \text{bodyPx}$) and extracts physical centimeter levers.
Module 07: Clinical Injury	ASIS Hips, Knees, Ankles	Calculates Munro FPPA ($<165^\circ$) for dynamic knee valgus and McGill spinal line ($<150^\circ$) for lumbar sag.

10. Scientific Standards, Google Research Benchmarks, & Citations

- **Bazarevsky, Grishchenko, Raveendran, Zhu, Zhang, & Grundmann (2020):**

"BlazePose: On-device Real-time Body Pose Tracking." Google Research, arXiv:2006.10204.

Key Finding: Established the two-stage detector-tracker pipeline achieving 33-landmark 3D pose topology at $>30\text{ FPS}$ on standard mobile smartphone CPUs.

- **Cao, Simon, Wei, & Sheikh (CVPR 2017):**

"Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields (OpenPose)."

Significance: Established the foundational mathematical concepts for Part Affinity Fields (PAFs) that inspired modern heatmap architectures.

- **International Society of Biomechanics (ISB Recommendations):**

Provides the standard anatomical joint definitions (acromion, lateral epicondyle, styloid process, greater trochanter, lateral malleolus) mirrored by the 33-point BlazePose topology.

11. Panel Defense Quick-Reference: Frequently Asked Questions & Rapid Answers

Q1: How can a standard smartphone camera track 33 joints in 3D space without requiring physical markers or wearable sensors?

Rapid Defense Answer: We utilize Google ML Kit BlazePose, a deep convolutional neural network trained on hundreds of thousands of diverse human action poses. The model uses heat-map regression to predict the 2D pixel coordinates (x, y) of 33 anatomical landmarks, and simultaneously predicts a relative depth coordinate (z) by learning perspective foreshortening and biomechanical bone constraints, achieving markerless 3D motion capture on raw video.

Q2: Why did you choose BlazePose over popular frameworks like OpenPose or YOLOv8-Pose?

Rapid Defense Answer: OpenPose is computationally massive, requiring high-power desktop GPUs ($>250\text{W}$), making it impossible to run locally on a smartphone at 30 FPS. While YOLOv8-Pose is fast, it only predicts 17 keypoints and completely lacks foot landmarks (heels and foot indices). BlazePose provides 33 landmarks including heels and toes, which are medically essential for detecting knee valgus and foot placement, while executing in $\sim 18\text{ ms}$ on mobile silicon.

Q3: How does BlazePose achieve 30 frames per second on a mobile phone without overheating the processor?

Rapid Defense Answer: It uses a two-stage Detector-Tracker architecture. Finding a human in a full video frame is computationally heavy, but tracking them once found is lightweight. The heavy detector runs only on the very first frame. In all subsequent frames, the system bypasses the detector and crops a small region of interest directly from the previous frame's joints, reducing computational load by over 80%.

Q4: What does the relative depth coordinate 'z' mean if the camera only has a flat 2D lens?

Rapid Defense Answer: Because a single lens cannot directly measure laser time-of-flight, the z coordinate represents relative depth calibrated to the midpoint between the athlete's hips ($z=0$). If a joint moves closer to the camera than the hips (such as punching an arm forward), z becomes negative. If it moves further away, z becomes positive. This enables true 3D spatial vector calculations invariant to 2D perspective.

Q5: What is the purpose of the YUV420 memory buffer concatenation in your code?

Rapid Defense Answer: Android cameras stream video in raw YUV420 format across three separate memory planes (luminance and two chrominance planes). Naive frame conversion in Dart creates massive garbage collection churn and freezes the UI. We engineered a native `WriteBuffer` that concatenates all byte planes into a contiguous NV21 byte buffer in memory in $<1\text{ millisecond}$, completely eliminating dropped frames.

Q6: What happens if an athlete's limb is blocked by exercise equipment or moves out of the camera frame?

Rapid Defense Answer: Every landmark includes a visibility confidence score (v_k). If $v_k < 0.45$, our engine classifies the joint as occluded and removes it from the rendering and kinematic buffers. Furthermore, our boundary validation rejects frames where feet or head are cut off by the screen edges, prompting the voice coach to say "Step back into frame".

Q7: How does Module 02 connect to the rest of the 9 modules in the system?

Rapid Defense Answer: Module 02 is the foundational sensory pipeline. It extracts 33 landmarks at 30 FPS and buffers them in a 90-frame queue. It feeds 48-frame window tensors to Module 03 (Action Recognition), joint flexion angles to Module 04 (Rep Counting), 3D spatial vectors to Module 05 (Posture Correctness), vertical body pixel spans to Module 06 (Body Scanner), and knee-hip-ankle lines to Module 07 (Clinical Injury Prevention).

